

12 · Pruebas y calidad

Documentación de sistema

Versión analizada: **0.1.1+2**

Commit: **a7ebf2c**

Fecha del documento: **2026-08-27**

Documento generado desde el Markdown de **docs/system-documentation/** con **tool/build_system_documentation_pdf.py**. El Markdown es la fuente: no editar este PDF.

Contenido

1. Qué se ejecutó y qué no, en este análisis
2. Tipos de prueba existentes
3. Inventario por área
4. Cobertura observable
5. Cómo ejecutar las pruebas
6. Datos y fixtures
7. Herramientas de análisis estático
8. Qué comprueba ``validate_structure.py``
9. Integración continua
10. Criterios de aceptación
11. Casos límite ya cubiertos
12. Propuesta priorizada de pruebas faltantes

Qué se ejecutó y qué no, en este análisis

Comprobado localmente:

Comando	Resultado
<code>python tool/verify_rootcause_contract.py</code>	Correcto: 26 reglas coherentes entre motor, esquema, textos, fixtures, política, versión y redacción
<code>python tool/validate_structure.py --require-lock</code>	Correcto
Recuento de casos sobre <code>test/</code> e <code>integration_test/</code>	88 declarados

No ejecutado localmente: `flutter analyze`, `flutter test` y las compilaciones. Flutter y Dart no están instalados en la máquina de análisis.

Comprobado en la CI pública, ejecución [33029927460](#) sobre el commit [a7ebf2c](#):

Gate	Resultado
Estructura y contrato offline	Correcto
<code>flutter analyze --fatal-infos</code>	Sin hallazgos
<code>flutter test --coverage</code>	Verde
<code>flutter build web --release</code>	Correcto
<code>flutter build apk --release</code>	Correcto
<code>flutter build ios --debug --simulator</code>	Correcto

Tipos de prueba existentes

Tipo	Archivos	Casos	Necesita dispositivo
Unitarias puras	15	55	No
De widget	5	15	No
Con base de datos en memoria	3	17	No
De integración	1	1	Sí

`flutter test` ejecuta los 87 de `test/`. El caso de `integration_test/` exige un dispositivo o emulador y no forma parte del gate de CI.

Inventario por área

Área	Archivo	Casos
Motor de reglas	<code>test/core/qr_investigation_engine3test.dart</code>	

Adaptador de riesgo	test/scan_security_analyzer_test.dart	9
Barra de estado y overlay	test/features/scan_status_bar_test.dart	1
Confirmación de lectura	test/services/scan_feedback_test.dart	6
Intérprete de contenido	test/content_interpreter_test.dart	6
Cifrado	test/core/payload_cipher_test.dart	5
Importación	test/services/import_service_test.dart	5
Evidencia	test/core/qr_evidence_exporter_test.dart	4
Registro de lectura	test/scan_record_test.dart	4
Localización	test/core/localization_test.dart	4
Marco de teléfono	test/core/handheld_frame_test.dart	4
Rotación de llave	test/core/data_maintenance_test.dart	3
Geometría del visor	test/features/scanner_viewport_geometry_test.dart	3
Configuración del motor de captura	test/features/scanner_engine_config_test.dart	3
Cola de escrituras	test/core/async_write_queue_test.dart	2
Cancelación y progreso	test/core/cancellation_token_test.dart	2
Migración de esquema	test/core/schema_migrator_test.dart	1
Recuperación	test/core/recovery_service_test.dart	1
Diagnóstico	test/core/diagnostics_test.dart	1
Accesibilidad	test/core/accessibility_test.dart	1
Historial	test/services/history_repository_test.dart	1
Registro de parsers	test/services/content_parser_registry_test.dart	1
Sesión de inventario	test/inventory_session_test.dart	1
Arranque	integration_test/app_launch_test.dart	1

Cobertura observable

La CI genera coverage/lcov.info y lo publica como artefacto, pero **no hay umbral de cobertura** ni informe agregado en el repositorio. Por eso este documento no cita un porcentaje: sería un número no verificado.

Lo que sí puede afirmarse leyendo el código y las pruebas:

Módulo	Cobertura de comportamiento	Comentario
QrInvestigationEngine	Alta	13 casos sobre las reglas de mayor peso, incluidos negativos

PayloadCipher	Alta	Los cinco caminos de fallo
QrEvidenceExporter	Alta	Redacción, alteración, canonicalización y validación de entrada
ImportService	Alta	Límites y el control anti-inyección de veredicto
DataMaintenanceService	Alta	Incluido el caso de llave huérfana
ContentInterpreter	Media	6 de 17 familias de contenido
ScanStatusBar	Media-alta	Cinco estados y movimiento reducido
ScanFeedback	Alta	Los dos canales y toda la degradación
ScanRecord	Media	Serialización, binario y sensibilidad
HistoryRepository	Baja	Solo replaceAll. Sin pruebas de la migración heredada
InventoryRepository	Ninguna directa	—
SettingsRepository	Ninguna	—
RecoveryService	Baja	Solo el paquete de export
ScannerScreen	Ninguna	Requiere cámara
InventoryScreen	Ninguna	Requiere cámara
HistoryScreen	Ninguna	—
GeneratorScreen	Ninguna	—
SettingsScreen	Ninguna	—
ScanResultsSheet	Ninguna	—
PdfPageRenderer	Ninguna	Requiere dart:io y un PDF
ExportService	Ninguna	Requiere la hoja de compartir
BiometricService	Ninguna	Requiere plataforma

Cómo ejecutar las pruebas

```
flutter test
```

```
flutter test --coverage
```

```
flutter test test/core/qr_investigation_engine_test.dart
```

```
flutter test integration_test/app_launch_test.dart
```

Verificadores offline, que no necesitan Flutter:

```
python tool/validate_structure.py --require-lock
python tool/verify_rootcause_contract.py
```

Datos y fixtures

Recurso	Qué contiene	Quién lo valida
<code>fixtures/qr/manifest.json</code>	12 casos sintéticos con carga, severidad, acción esperadas y reglas que deben o no aparecer	<code>verify_rootcause_contract.py</code>
<code>test_assets/manifest.json</code>	Imágenes reales de regresión por categoría	<code>validate_structure.py</code>
<code>test_assets/qr/,wifi,damaged,low_light,gs1,multiple_codes</code>	PNG reales	—

Reglas que el verificador impone a los fixtures:

- mínimo 12 casos, con id único;
- acción y severidad dentro de los valores válidos;
- toda regla citada debe existir en el motor;
- **hosts reservados obligatorios:** `example`, `example.com` o `192.168.*`, con `bit.ly` como única excepción, necesaria para probar la regla de acortadores;
- once reglas concretas deben tener al menos un fixture que las cubra.

Hallazgo. Los 12 fixtures **no los consume ninguna prueba Dart**. Los valida el verificador Python, que comprueba su forma y su coherencia con el motor, pero nadie ejecuta el motor contra ellos y compara el resultado con `expectedSeverity` y `expectedAction`. Es la oportunidad de mejora más rentable del repositorio. Registrado en [15-risks-and-technical-debt.md](#).

Lo mismo ocurre con `test_assets/`: las imágenes existen y su manifiesto se valida, pero ninguna prueba las decodifica —haría falta el motor nativo de la plataforma—.

Herramientas de análisis estático

Herramienta	Configuración	En CI
<code>flutter analyze --fatal-infos</code>	<code>analysis_options.yaml</code>	Sí, bloqueante
<code>flutter_lints 6.0.0</code>	Base	Sí
Reglas propias	<code>always_declare_return_types</code> , <code>avoid_print</code> , <code>prefer_final_locals</code> , <code>use_build_context_synchronously</code>	Sí
<code>validate_structure.py</code>	14 comprobaciones	Sí, bloqueante
<code>verify_rootcause_contract.py</code>	Contrato de reglas y evidencia	Sí, bloqueante

`--fatal-infos` convierte cualquier sugerencia en error. Es un gate estricto: en la primera ejecución de CI de la versión 0.1.1, un único `undefined_class` tumbó todo el pipeline, que es exactamente lo que debe ocurrir.

Las cuatro reglas propias tienen intención:

- `avoid_print` impide que una carga escaneada acabe en la consola;
- `use_build_context_synchronously` evita usar un contexto tras un `await`, un fallo frecuente en pantallas con diálogos;
- `always_declare_return_types` y `prefer_final_locals` sostienen la legibilidad.

Qué comprueba `validate_structure.py`

Comprobación	Qué previene
<code>check_yaml</code>	YAML inválido en manifiesto y workflows
<code>check_action_pins</code>	Acciones sin fijar a SHA de commit
<code>check_imports</code>	Import local a un archivo inexistente
<code>check_absolute_paths</code>	Rutas del entorno de desarrollo filtradas al repositorio
<code>check_version</code>	Formato de versión inválido
<code>check_interface_version</code>	Que la interfaz muestre una versión distinta de la construida
<code>check_sensitive_logging</code>	Registro de contenido sensible
<code>check_test_assets</code>	Manifiesto de regresión que apunte a un archivo inexistente
<code>check_json_files</code>	JSON inválido
<code>check_markdown_links</code>	Enlace Markdown local roto
<code>check_landing</code>	Recurso de la landing inexistente o no ensamblado por el workflow
<code>check_launcher_assets</code>	Icono de producto ausente o vacío
<code>check_mobile_product_scope</code>	Que el escritorio vuelva a presentarse como plataforma del producto
<code>check_source_sbom</code>	SBOM de fuente ausente o no CycloneDX

Integración continua

Tres workflows; detalle en [13-deployment-and-operations.md](#).

Workflow	Disparo	Bloquea
<code>flutter-ci.yml</code>	push a <code>main/develop</code> , PR, manual	Sí
<code>android-release.yml</code>	tag <code>vX.Y.Z</code>	Sí: analiza y prueba antes de compilar
<code>deploy-landing.yml</code>	cambios publicables en <code>main</code>	No al código

Que el workflow de release **repita** análisis y pruebas antes de compilar es deliberado: garantiza que ningún artefacto se publique desde un árbol que no pasa el gate.

Criterios de aceptación

Los que aplica la CI hoy:

1. estructura y contrato offline correctos;
2. `flutter analyze --fatal-infos` sin hallazgos;
3. `flutter test` en verde;
4. compilan Android, web e iOS simulador;
5. para un release, la versión del tag coincide con `pubspec.yaml`.

Los que **no** aplica y están declarados como pendientes:

6. matriz de dispositivos físicos —ver `../quality/DEVICE_TEST_MATRIX.md`;
7. accesibilidad con lector de pantalla real;
8. umbral de cobertura;
9. auditoría de seguridad independiente.

Casos límite ya cubiertos

Vale la pena conocerlos, porque son los que documentan decisiones sutiles:

Caso	Prueba
Un dominio que empieza por <code>fc</code> no es una IPv6 privada	motor
Un correo en la consulta no es una credencial	analizador
La huella cubre la carga exacta, con espacios y controles	motor
Un control codificado bloquea también un <code>mailto:</code>	motor
El checksum de evidencia resiste reordenar las claves	evidencia
Un enlace anterior mal formado se rechaza	evidencia
Una escritura fallida no bloquea las siguientes	cola
Un idioma no soportado no produce interfaz a medias	localización
Exactamente en el umbral de 560 px la pantalla se llena	marco
Con movimiento reducido la barra es legible sin animación	barra de estado
Un lector de pantalla distingue captura de pausa	barra de estado
Un reproductor de audio que falla degrada y deja de intentarlo	confirmación

Propuesta priorizada de pruebas faltantes

Ordenada por relación entre riesgo cubierto y esfuerzo.

Prioridad 1 — riesgo alto, esfuerzo bajo

Prueba	Por qué
Ejecutar los 12 fixtures contra el motor y comparar severidad, acción, mustInclude y mustExclude	El activo de regresión existe y nadie lo usa. Es la brecha más grande
HistoryRepository: migración heredada completa	El procedimiento con más riesgo de pérdida de datos no tiene prueba propia
HistoryRepository: registro ilegible se aísla y el resto carga	Propiedad de resiliencia central
HistoryRepository: recorte a maxItems y poda por retención	Borran datos de la persona
SettingsRepository: valores por defecto y resetNonSensitive	Se apoya en ella el arranque seguro

Prioridad 2 — riesgo alto, esfuerzo medio

Prueba	Por qué
InventoryStore con la cola: lecturas y ediciones concurrentes	La cola tiene prueba; su uso real no
ScanResultsSheet: block no muestra botón; confirm exige diálogo	Es una propiedad de seguridad verificada hoy solo por un grep del verificador
ScanResultsSheet: la ocultación de sensibles funciona	Control de exposición
ContentInterpreter: las 11 familias sin cobertura, en especial AAMVA, Swiss QR y EPC	Deciden sensitive
ScanRecord.fromJson con trustDerivedAnalysis en ambos valores	Control anti-inyección

Prioridad 3 — completitud

Prueba	Por qué
ExportService: cabeceras y escapado del CSV	Un carácter mal escapado corrompe el archivo
PdfPageRenderer: cancelación y limpieza	Requiere un PDF de prueba
RecoveryService.retry y discard	Solo el export tiene prueba
Widget del historial: filtros y búsqueda	—
Umbral de cobertura en CI	Evita regresiones silenciosas

Prioridad 4 — requiere dispositivo

Toda la matriz de `../quality/DEVICE_TEST_MATRIX.md`, con las cuatro filas añadidas en 0.1.1: código lejano y descentrado, confirmación de captura, relectura del mismo código y conteo de unidades repetidas.